

1 Introduction

Business Process Model and Notation (BPMN) is a widely used modeling language for specifying business processes. It provides a standardized graphical notation that is easily understandable by all business stakeholders, including business analysts, technical developers, and business managers. However, while BPMN is powerful, it has certain limitations.

BPMN excels at modeling business processes through its various flow objects such as events, activities, and gateways. It allows for the detailed representation of tasks and sub-processes, uses swimlanes to show different participants or roles involved in a process, and includes artifacts and data objects to add context to the processes. Furthermore, BPMN models can be directly executed using BPM engines, facilitating the automation of business processes. It also supports complex process control through various event types (e.g., start, intermediate, and end events) and gateways (e.g., exclusive, inclusive, parallel) for controlling the divergence and convergence of sequence flows.

Despite these strengths, BPMN has limitations. It does not inherently support modeling cross-cutting concerns like security, compliance, and regulatory requirements, which often need to be integrated into the process through custom annotations or external specifications. BPMN also lacks a native mechanism to specify behavioral constraints and scenarios, such as negative scenarios (scenarios that should not happen), mandatory scenarios (scenarios that must happen), and optional scenarios (scenarios that may occur). Additionally, while BPMN can model basic events, it does not natively support complex event processing or specify conditions based on complex combinations of events. Moreover, BPMN focuses on the flow of activities but provides limited capabilities for modeling detailed roles and responsibilities, especially in dynamic or highly collaborative environments.

To address these limitations, we propose BFlow, a companion modeling language to BPMN that allows specifying cross-cutting aspects of behavior. BFlow enables the specification of negative scenarios, enhancing regulatory and security compliance by identifying undesirable scenarios that need to be avoided. It supports defining mandatory scenarios that must occur upon certain triggers, ensuring critical business rules and requirements are enforced. Additionally, BFlow can model potential scenarios that may occur under certain conditions, providing a more flexible and realistic representation of business processes. BFlow can be used alongside BPMN models to refine and enhance them, providing a comprehensive approach to modeling complex business processes with additional behavioral constraints.

By combining BPMN with BFlow, more robust and compliant business process models can be created that not only define the flow of activities but also specify critical behavioral constraints. This integration is particularly useful for incorporating regulatory and security aspects into BPMN specifications, ensuring that business processes adhere to required standards and behaviors.

2 Related Work

The limitations of BPMN in handling cross-cutting concerns have led to the development of various extensions and alternative approaches. Among these, Aspect-Oriented Business Process Modeling Notation (AO-BPMN) and its successor AO-BPMN 2.0 have been significant contributions.

AO-BPMN and AO-BPMN 2.0: AO-BPMN was introduced to address the challenge of modeling cross-cutting concerns in business processes. Cross-cutting concerns, such as security, logging, and error handling, often permeate multiple process models, making them difficult to manage and modify. AO-BPMN provides mechanisms to modularize these concerns, separating them from the core process flow. AO-BPMN 2.0 further refines these concepts, enhancing readability, graphical scalability, and simplicity [1, 2]. Despite these improvements, AO-BPMN languages have limitations in capturing complex behaviors that involve intricate interactions and dependencies between different process elements.

Behavioral Programming: Behavioral programming (BP) is an approach to software development that enables incremental development by interweaving independent behavior threads at runtime. Each thread represents a scenario that the system should or should not follow. This approach allows for the specification of complex behaviors and interactions in a natural and modular way [3]. BPFLOW, an implementation of behavioral programming, leverages this approach to provide a more flexible and comprehensive framework for modeling business processes.

BPFLOW builds on the principles of behavioral programming to address the limitations of AO-BPMN languages. While AO-BPMN can modularize cross-cutting concerns, it struggles with scenarios that involve complex behaviors and dynamic interactions. BPFLOW, on the other hand, excels in specifying negative, mandatory, and optional scenarios, providing a more detailed and realistic representation of business processes. By integrating BPFLOW with BPMN, it is possible to create robust and compliant business process models that not only define the flow of activities but also specify critical behavioral constraints.

3 Examples

4 The BFLOW language

Definition 1. A *token*, denoted τ , is a function $\tau: Var \rightarrow Val$ that maps variable names to their values, where Var is the set of variable names used in the token and Val is the set of possible values that can be assigned to variables. We assume that $\{Request, WaitFor, Block\} \subseteq Var$ and that $\tau(v) \subseteq Events$ for each $v \in \{Request, WaitFor, Block\}$, marking the set of events that the token requests, waits-for, and blocks, respectively.

Definition 2. A *BFLOW diagram* is defined by the triplet (B, E, I) where:

- B is a set of blocks, where each block $b \in B$ has:
 - **Input port** $b.p_{in}$: Zero or one input port.
 - **Output ports** $b.p_{out}$: A set of output ports, including the special port *sync*.
 - **Transformation function** $b.trn$: $\mathcal{T} \rightarrow \mathcal{P}(\mathcal{T} \times b.p_{out})$: A function that maps input tokens to sets of output tokens at each output port.
- $E \subseteq (\bigcup_{b \in B} b.p_{out}) \times (\bigcup_{b \in B} b.p_{in})$ represents the set of directed edges, where each edge $(p, b'.p_{in}) \in E$ denotes a connection from output port p of block b to input port $b'.p_{in}$ of block b' .
- $I \subseteq \mathcal{T} \times B$ is the set of initial tokens.

Definition 3. The **transition system** $T(\tau_0, b_0)$ associated with a token τ_0 at block b_0 in a BFlow diagram (B, E, I) is defined as follows:

- The states are elements of $\mathcal{T} \times B$, where $\mathcal{T}$ is the set of all possible tokens and B is the set of blocks in the diagram.
- The initial state is (τ_0, b_0) , indicating that the token τ_0 starts at block b_0 .
- Step transitions are defined by the rule:

$$\frac{(\tau', p) \in b.trn(\tau) \quad \text{and} \quad (p, b'.p_{in}) \in E \quad \text{and} \quad p \neq \text{sync}}{(\tau, b) \rightarrow (\tau', b')}$$

This says that if applying the transformation function $b.trn$ to token τ yields token τ' with output at port p , and there exists an edge in E from port p of block b to input port $b'.p_{in}$ of block b' , and p is not the special port 'sync', then the token τ transitions to τ' at block b' .

- Sync transitions are defined by the rules:

$$\frac{(\tau', \text{sync}) \in b.trn(\tau)}{(\tau, b) \rightarrow (\tau', \text{sync}(b))} \quad \frac{}{(\tau, \text{sync}(b)) \rightarrow (\tau, \text{sync}(b))}$$

This says that if applying the transformation function $b.trn$ to token τ yields token τ' with output at the special port 'sync', then the token τ transitions to τ' at the 'sync' state and remains there.

Definition 4. A **run** of a set of BFlow diagrams is a sequence $S_0, S_1, \dots$ where:

- S_0 is a set containing the initial states of the transition systems of the initial tokens of the BFlow diagrams.
- If S_i contains a pair not of the form $(\tau, \text{sync}(b))$, where τ is a token and b is a block:

$$S_{i+1} = \{(\tau', b') : (\tau, b) \in S_i \text{ and } (\tau, b) \rightarrow (\tau', b')\}.$$

- If $S_i = \{(\tau_1, \text{sync}(b_1)), \dots, (\tau_n, \text{sync}(b_n))\}$:

$$S_{i+1} = \{(\tau[\text{event} \mapsto e], b) : (\tau, \text{sync}(b)) \in S_i \text{ and } e \in \tau(\text{WaitFor}) \cup \tau(\text{Request})\}$$

$$\cup$$

$$\{(\tau, \text{sync}(b)) : (\tau, \text{sync}(b)) \in S_i \text{ and } e \notin \tau(\text{WaitFor}) \cup \tau(\text{Request})\}$$
for some
$$e \in \bigcup_{i=1}^n \tau_i(\text{Request}) \setminus \bigcup_{i=1}^n \tau_i(\text{Block}).$$

Definition 5. A **SYNC-channel** is a type of channel in a Channel System where:

- Process P_i transmits a sync statement to the channel, and he can do it only if all the other processes also transmit to this channel.
- Then there will be an event trigger and the channel will return this event to all processes that wait or request this event.
- We define the next translation role from this channel to the Transaction System:

$$\frac{\forall_i \text{Loc}_i \xrightarrow{\text{sync}(\text{Loc}_i, \eta)! \text{SYNC}} \text{Loc}'_i \wedge \exists e. e \in \bigcup_{i=1}^n \text{Request}_i \setminus \bigcup_{i=1}^n \text{Block}_i}{(\text{Loc}_1, \text{Loc}_2, \dots, \text{Loc}_n, \eta) \xrightarrow{\text{true}:e} (f(\text{Loc}_1), f(\text{Loc}_2), \dots, f(\text{Loc}_n), \eta')}$$

where:

$$f(\text{Loc}_i) = \begin{cases} \text{Loc}'_i & \text{if } e \in \text{Request}_i \cup \text{Wait}_i \\ \text{Loc}_i & \text{otherwise} \end{cases}$$

References

- [1] R. M. Pillat, R. M. S. Santos, and T. C. Oliveira, “Systematic literature review on bpmn-based process adaptation approaches,” in *Proceedings of the XV Brazilian Symposium on Information Systems*, ser. SBSI '19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: <https://doi.org/10.1145/3330204.3330242>
- [2] L. P. Carvalho, C. Cappelli, and F. M. Santoro, “Ao-bpm 2.0: Aspect oriented business process modeling,” in *Business Process Management Workshops*, E. Teniente and M. Weidlich, Eds. Cham: Springer International Publishing, 2018, pp. 719–731.
- [3] D. Harel, A. Marron, and G. Weiss, “Programming coordinated behavior in Java,” in *Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*, vol. 6183 LNCS, 2010, pp. 250–274. [Online]. Available: http://link.springer.com/chapter/10.1007/978-3-642-14107-2_12